



Tecnológico de Monterrey

Inteligencia artificial avanzada para la ciencia de datos (Gpo 601)

**Momento de Retroalimentación: Módulo 2 Implementación de
una técnica de aprendizaje máquina sin el uso de un framework.
(Portafolio Implementación)**

Carlos Manuel Delgado Vázquez

Matrícula: A01747037

IDM

Repositorio: https://github.com/CDV-97/M2_ML.git

Objetivo

El objetivo de este proyecto fue implementar manualmente un algoritmo de machine learning sin utilizar frameworks para construir el modelo.

Se desarrolló un Árbol de Decisión para clasificación, programando su lógica principal desde cero, incluyendo la creación de nodos, cálculo de impureza Gini, selección de divisiones, crecimiento recursivo del árbol y generación de predicciones. Para comprobar su funcionamiento, el algoritmo fue probado con el dataset Water Potability.

Dataset utilizado

Se utilizó el dataset **Water Potability**, obtenido de Kaggle, contiene 3,276 observaciones y 10 columnas, incluyendo la variable objetivo.

El objetivo del dataset es clasificar muestras de agua según su potabilidad:

- 0: agua no potable
- 1: agua potable

Las variables utilizadas fueron:

- ph
- Hardness
- Solids
- Chloramines
- Sulfate
- Conductivity
- Organic_carbon
- Trihalomethanes
- Turbidity

La variable objetivo fue:

- Potability

El dataset contiene algunos valores faltantes. Para poder utilizar todas las observaciones, los valores faltantes fueron reemplazados con la media de cada variable.

División de entrenamiento y prueba

Los datos fueron separados de la siguiente manera:

- **80% para entrenamiento**
- **20% para prueba**

Se utilizó:

```
Python  
random_state=42
```

para obtener resultados reproducibles.

También se utilizó:

```
Python  
stratify=y
```

para mantener aproximadamente la misma proporción de las clases 0 y 1 tanto en entrenamiento como en prueba. El conjunto de prueba estuvo compuesto por 656 observaciones.

Las herramientas de scikit-learn se utilizaron únicamente para dividir los datos y evaluar resultados. El modelo de Árbol de Decisión no utiliza ninguna implementación de Machine Learning de dicha biblioteca.

Implementación del Árbol de Decisión

El algoritmo fue implementado manualmente en el archivo:

```
Python  
decision_tree.py
```

La implementación incluye los siguientes elementos:

Nodo

Cada nodo almacena:

- variable utilizada para dividir,
- valor de corte o threshold,

- hijo izquierdo,
- hijo derecho,
- valor de clase en caso de ser una hoja.

Impureza Gini

Para evaluar qué tan buena es una división, se utiliza la impureza Gini:

$$Gini = 1 - \sum p_i^2$$

donde p_i representa la proporción de observaciones pertenecientes a cada clase. Un valor bajo de Gini indica que el grupo contiene principalmente observaciones de una sola clase.

Selección de la mejor división

El algoritmo prueba diferentes características y valores de corte. Para cada posible división se calcula el Gini ponderado de los grupos izquierdo y derecho. La división que produce el menor valor es seleccionada.

Crecimiento recursivo

El árbol continúa creando nuevas divisiones hasta que se cumple alguna de las condiciones de parada:

- se alcanza la profundidad máxima
- no existen suficientes observaciones para dividir
- todas las observaciones pertenecen a la misma clase
- no se encuentra una división válida.

Cuando ocurre alguna de estas condiciones, se genera un nodo hoja con la clase mayoritaria.

Predicción

Para realizar una predicción, cada observación recorre el árbol desde la raíz. Dependiendo del valor de la característica utilizada en cada nodo, la observación continúa por la rama izquierda o derecha hasta llegar a una hoja. La clase almacenada en esa hoja corresponde a la predicción final.

Configuración utilizada

Para la ejecución final se utilizó:

```
Python
DecisionTree(
    max_depth=3,
    min_samples_split=2
)
```

El modelo fue entrenado exclusivamente con el conjunto de entrenamiento y posteriormente evaluado con el conjunto de prueba.

Prueba de ejecución

El proyecto puede ejecutarse directamente mediante:

```
Python
main.py
```

Al ejecutarse, el programa carga el dataset, realiza el preprocesamiento, divide los datos en entrenamiento y prueba, entrena el Árbol de Decisión implementado manualmente, genera predicciones y calcula las métricas de evaluación.

```
Ejemplos de predicciones:
Ejemplo 1: Real = 1, Predicción = 0
Ejemplo 2: Real = 0, Predicción = 0
Ejemplo 3: Real = 0, Predicción = 0
Ejemplo 4: Real = 0, Predicción = 0
Ejemplo 5: Real = 0, Predicción = 0

Accuracy:
0.6326219512195121

Matriz de confusión:
[[374  26]
 [215  41]]

Reporte de clasificación:
      precision    recall  f1-score   support

     0       0.63      0.94      0.76       400
     1       0.61      0.16      0.25       256

 accuracy          0.62
 macro avg         0.62      0.55      0.51       656
weighted avg         0.63      0.63      0.56       656
```

Resultados

El modelo obtuvo un accuracy aproximado de:

Accuracy

$$\text{Accuracy} = 0.6326$$

Es decir, aproximadamente 63.2% de las observaciones del conjunto de prueba fueron clasificadas correctamente.

Matriz de confusión

La matriz de confusión obtenida fue:

```
Python
[[ 374  26]
 [ 215  41]]
```

Interpretación:

- **374** muestras de clase 0 fueron correctamente clasificadas como no potables.
- **26** muestras de clase 0 fueron clasificadas incorrectamente como potables.
- **215** muestras de clase 1 fueron clasificadas incorrectamente como no potables.
- **41** muestras de clase 1 fueron correctamente clasificadas como potables.

Métricas de clasificación

Los resultados por clase fueron aproximadamente:

Clase	Precision	Recall	F1-score
0	0.63	0.94	0.76
1	0.61	0.16	0.25

Para la clase 0, el modelo presenta un recall alto de **0.94**, por lo que logra identificar correctamente la mayoría de las muestras no potables.

Para la clase 1, el recall es de **0.16**, lo que significa que el modelo tiene mayor dificultad para detectar correctamente las muestras de agua potable.

Análisis de resultados

Los resultados muestran que el Árbol de Decisión implementado es capaz de aprender patrones del conjunto de entrenamiento y generar predicciones sobre datos que no fueron utilizados durante su entrenamiento.

El modelo presenta un mejor desempeño para la clase de agua no potable que para la clase potable. Esto puede observarse principalmente en el recall de la clase 0, que es considerablemente mayor al de la clase 1.

Aunque el desempeño general del modelo es limitado, el objetivo principal de la actividad no consiste en maximizar la precisión, sino en demostrar la implementación funcional de una técnica de Machine Learning sin utilizar un modelo previamente construido por un framework. El algoritmo desarrollado cumple con este objetivo, ya que toda la lógica del Árbol de Decisión fue programada manualmente.

Conclusión

Se logró implementar correctamente un Árbol de Decisión de clasificación desde cero. La implementación incluye el cálculo de impureza Gini, búsqueda de divisiones, crecimiento recursivo, creación de nodos hoja y predicción de nuevas observaciones.

El modelo fue probado utilizando el dataset Water Potability y obtuvo aproximadamente 63.2% de accuracy en el conjunto de prueba. Aunque existen diferencias importantes en el desempeño entre ambas clases, el modelo consigue separar los datos y realizar predicciones correctamente.

Por lo tanto, se considera que el objetivo del proyecto fue cumplido: desarrollar y ejecutar un algoritmo de aprendizaje máquina funcional sin utilizar un framework de Machine Learning para la implementación del modelo.

Link del Dataset

<https://www.kaggle.com/datasets/adityakadiwal/water-potability>